2 the Observer Pattern

Keeping your Objects in the Know



You don't want to miss out when something interesting happens, do you? We've got a pattern that keeps your objects *in the know* when something they *care about* happens. It's the Observer Pattern. It is one of the most commonly used design patterns, and it's incredibly useful. We're going to look at all kinds of interesting aspects of Observer, like its *one-to-many relationships* and *loose coupling*. And, with those concepts in mind, how can you help but be the life of the Patterns Party?

Congratulations!

Your team has just won the contract to build Weather-O-Rama, Inc.'s next-generation, internet-based Weather Monitoring Station.



Weather-O-Rama, Inc.
100 Main Street
Tornado Alley, OK 45021

Statement of Work

Congratulations on being selected to build our next-generation, internet-based Weather Monitoring Station!

The weather station will be based on our patent pending WeatherData object, which tracks current weather conditions (temperature, humidity, and barometric pressure). We'd like you to create an application that initially provides three display elements: current conditions, weather statistics, and a simple forecast, all updated in real time as the WeatherData object acquires the most recent measurements.

Further, this is an expandable weather station. Weather-O-Rama wants to allow other developers to write their own weather displays and plug them right in. So it's important that new displays will be easy to add in the future.

Weather-O-Rama thinks we have a great business model: once the customers are hooked, we intend to charge them for each display they use. Now for the best part: we are going to pay you in stock options.

We look forward to seeing your design and alpha application.

Sincerely,

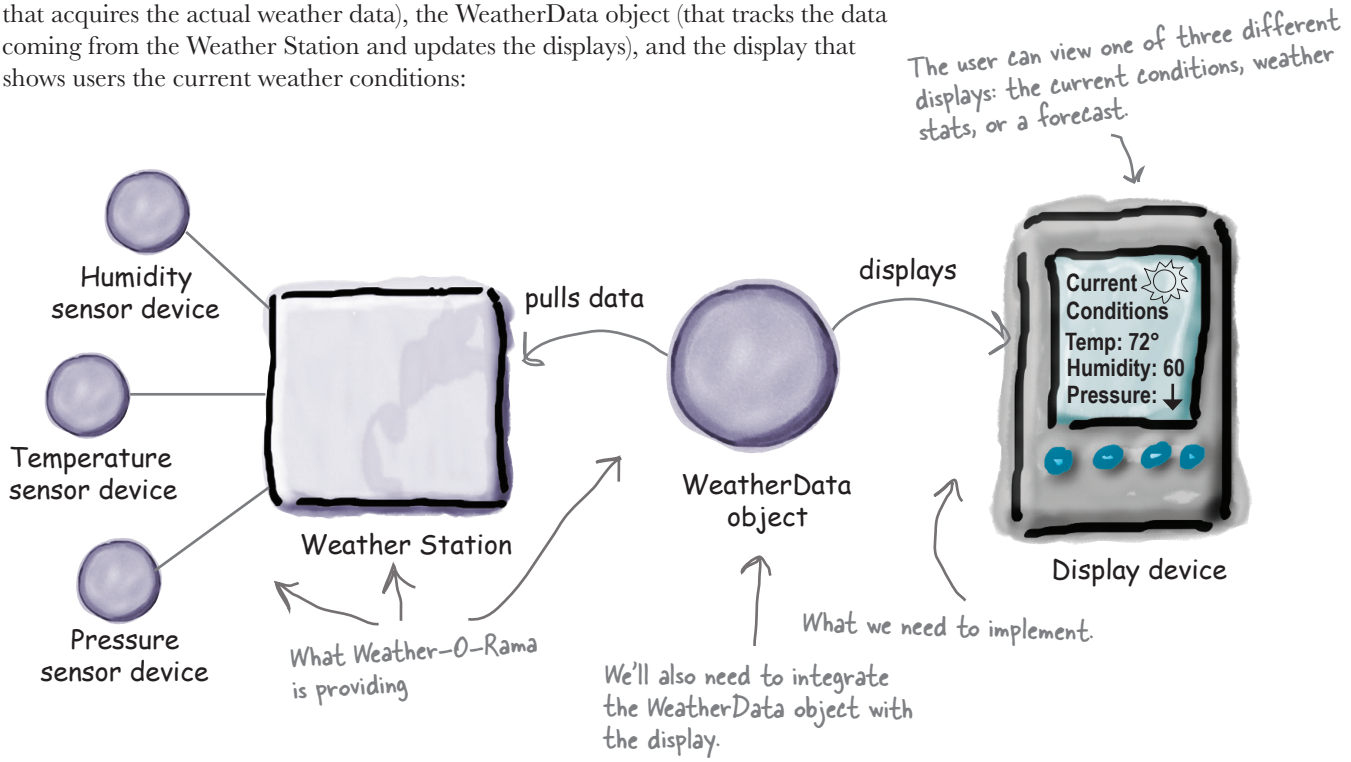
Johnny Hurricane

Johnny Hurricane, CEO

P.S. See the attached WeatherData source files!

The Weather Monitoring application overview

Let's take a look at the Weather Monitoring application we need to deliver—both what Weather-O-Rama is giving us, and what we're going to need to build or extend. The system has three components: the weather station (the physical device that acquires the actual weather data), the WeatherData object (that tracks the data coming from the Weather Station and updates the displays), and the display that shows users the current weather conditions:



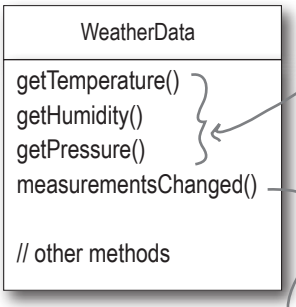
The WeatherData object was written by Weather-O-Rama and knows how to talk to the physical Weather Station to get updated weather data. We'll need to adapt the WeatherData object so that it knows how to update the display. Hopefully Weather-O-Rama has given us hints for how to do this in the source code. Remember, we're responsible for implementing three different display elements: Current Conditions (shows temperature, humidity, and pressure), Weather Statistics, and a simple Forecast.

So, our job, if we choose to accept it, is to create an app that uses the WeatherData object to update three displays for current conditions, weather stats, and a forecast.

Unpacking the WeatherData class

Let's check out the source code attachments that Johnny Hurricane, the CEO, sent over. We'll start with the WeatherData class:

Here is our WeatherData class.



These three methods return the most recent weather measurements for temperature, humidity, and barometric pressure, respectively.

We don't care right now HOW it gets this data, we just know that the WeatherData object gets updated info from the Weather Station.

Note that whenever WeatherData has updated values, the measurementsChanged() method is called.

Let's looks at the measurementsChanged() method, which, again, gets called anytime the WeatherData obtains new values for temp, humidity, and pressure.

```
/*
 * This method gets called
 * whenever the weather measurements
 * have been updated
 *
 */
public void measurementsChanged() {
    // Your code goes here
}
```

WeatherData.java

It looks like Weather-O-Rama left a note in the comments to add our code here. So perhaps this is where we need to update the display (once we've implemented it)

Our soon-to-be-implemented display.



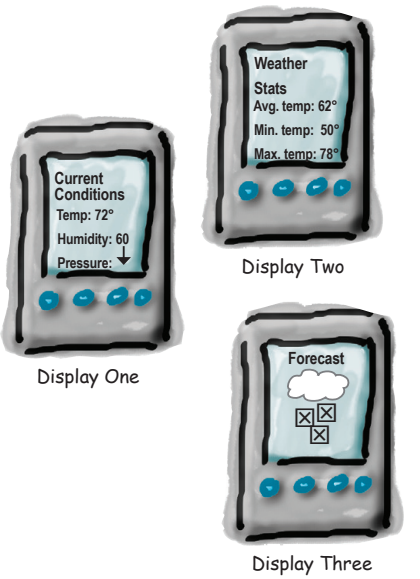
Display device

So, our job is to alter the measurementsChanged() method so that it updates the three displays for current conditions, weather stats, and forecast.

Our Goal

We know we need to implement a display and then have the WeatherData update that display each time it has new values, or, in other words, each time the measurementsChanged() method is called. But how? Let's think through what we're trying to acheive:

- We know the WeatherData class has getter methods for three measurement values: temperature, humidity, and barometric pressure.
- We know the measurementsChanged() method is called anytime new weather measurement data is available. (Again, we don't know or care how this method is called; we just know that it *is called.*)
- We'll need to implement three display elements that use the weather data: a *current conditions* display, a *statistics display,* and a *forecast* display. These displays must be updated as often as the WeatherData has new measurements.
- To update the displays, we'll add code to the measurementsChanged() method.



Stretch Goal

But let's also think about the future—remember the constant in software development? Change. We expect, if the Weather Station is successful, there will be more than three displays in the future, so why not create a marketplace for additional displays? So, how about we build in:

- Expandability—other developers may want to create new custom displays. Why not allow users to add (or remove) as many display elements as they want to the application? Currently, we know about the initial *three* display types (current conditions, statistics, and forecast), but we expect a vibrant marketplace for new displays in the future.



Taking a first, misguided implementation of the Weather Station

Here's a first implementation possibility—as we've discussed, we're going to add our code to the measurementsChanged() method in the WeatherData class:

```
public class WeatherData {

    // instance variable declarations

    public void measurementsChanged() {

        float temp = getTemperature();
        float humidity = getHumidity();
        float pressure = getPressure();

        currentConditionsDisplay.update(temp, humidity, pressure);
        statisticsDisplay.update(temp, humidity, pressure);
        forecastDisplay.update(temp, humidity, pressure);
    }

    // other WeatherData methods here
}
```

Here's the measurementsChanged() method.

And here are our code additions...

First, we grab the most recent measurements by calling the WeatherData's getter methods. We assign each value to an appropriately named variable.

Next we're going to update each display...

...by calling its update method and passing it the most recent measurements.



Sharpen your pencil

Based on our first implementation, which of the following apply? (Choose all that apply.)

- ❑ A. We are coding to concrete implementations, not interfaces.
- ❑ B. For every new display we'll need to alter this code.
- ❑ C. We have no way to add (or remove) display elements at runtime.
- ❑ D. The display elements don't implement a common interface.
- ❑ E. We haven't encapsulated the part that changes.
- ❑ F. We are violating encapsulation of the WeatherData class.

What's wrong with our implementation anyway?

Think back to all those Chapter 1 concepts and principles—which are we violating, and which are we not? Think in particular about the effects of change on this code. Let's work through our thinking as we look at the code:

```
public void measurementsChanged() {

    float temp = getTemperature();
    float humidity = getHumidity();
    float pressure = getPressure();

    currentConditionsDisplay.update(temp, humidity, pressure);
    statisticsDisplay.update(temp, humidity, pressure);
    forecastDisplay.update(temp, humidity, pressure);
}
```

Let's take another look...

Looks like an area of change. We need to encapsulate this.

By coding to concrete implementations, we have no way to add or remove other display elements without making changes to the code.

At least we seem to be using a common interface to talk to the display elements...they all have an update() method that takes the temp, humidity, and pressure values.

What if we want to add or remove displays at runtime? This looks hardcoded.



Good idea. Let's take a look at Observer, then come back and figure out how to apply it to the Weather Monitoring app.

Meet the Observer Pattern

You know how newspaper or magazine subscriptions work:

1. A newspaper publisher goes into business and begins publishing newspapers.

2. You subscribe to a particular publisher, and every time there's a new edition it gets delivered to you. As long as you remain a subscriber, you get new newspapers.

3. You unsubscribe when you don't want papers anymore, and they stop being delivered.

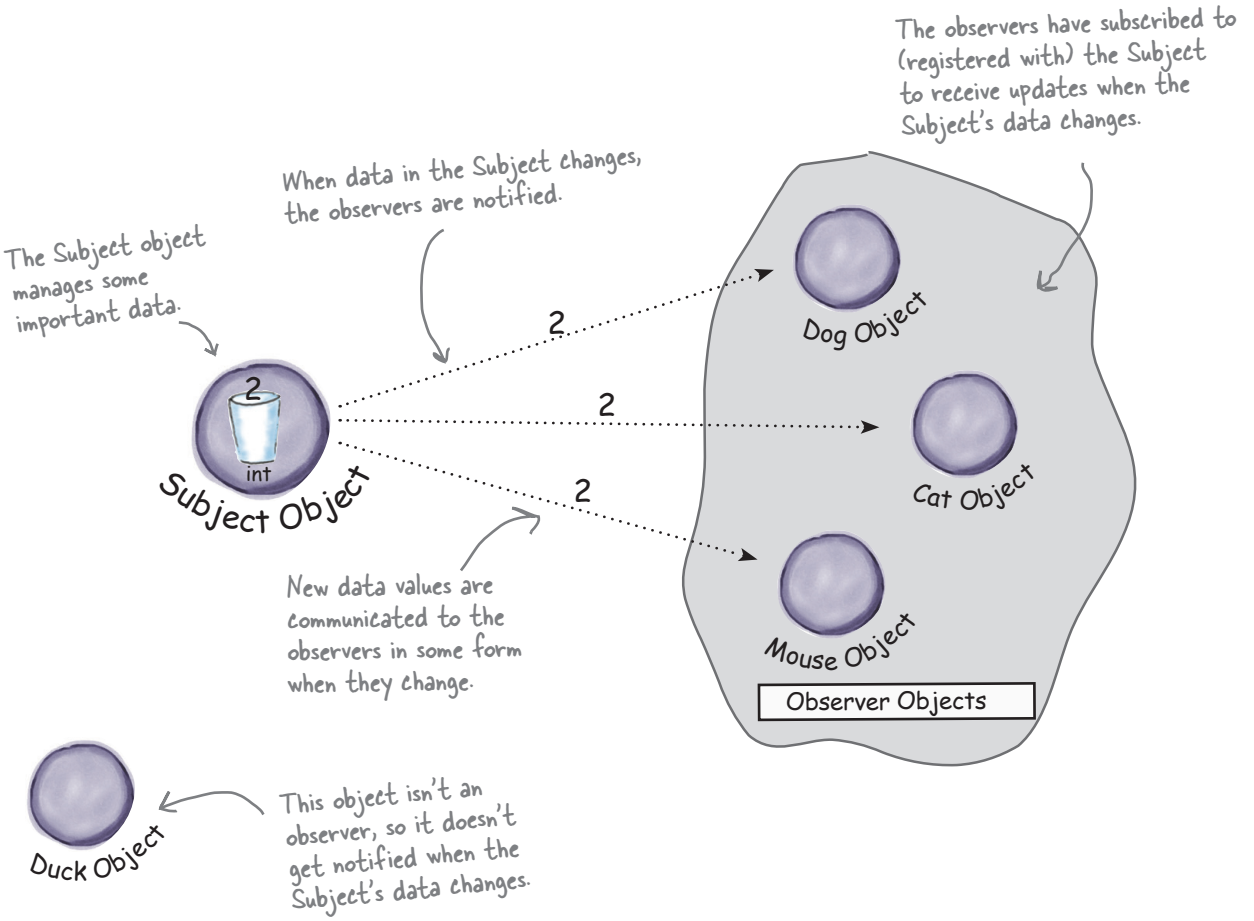
4. While the publisher remains in business, people, hotels, airlines, and other businesses constantly subscribe and unsubscribe to the newspaper.



Publishers + Subscribers = Observer Pattern

If you understand newspaper subscriptions, you pretty much understand the Observer Pattern, only we call the publisher the SUBJECT and the subscribers the OBSERVERS.

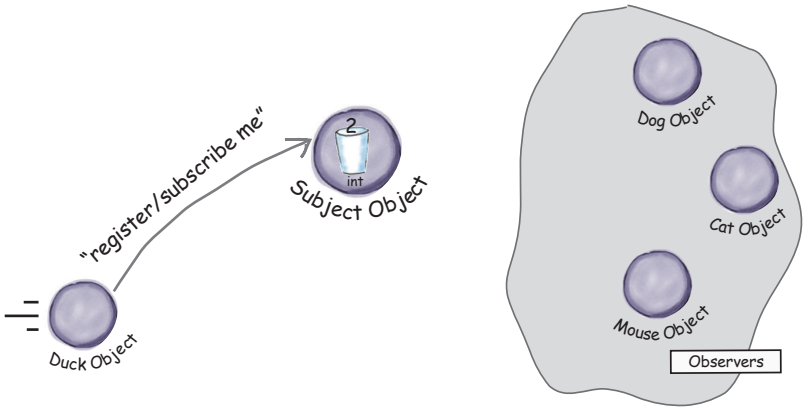
Let's take a closer look:



A day in the life of the Observer Pattern

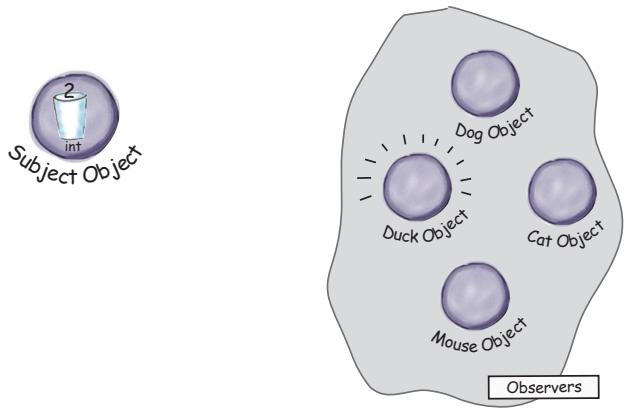
A Duck object comes along and tells the Subject that he wants to become an observer.

Duck really wants in on the action; those ints Subject is sending out whenever its state changes look pretty interesting...



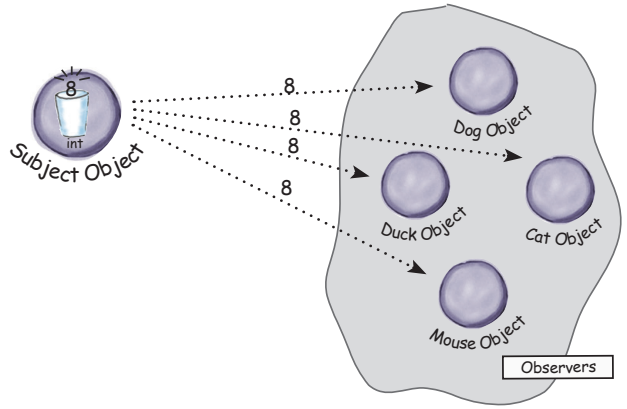
The Duck object is now an official observer.

Duck is psyched...he's on the list and is waiting with great anticipation for the next notification so he can get an int.



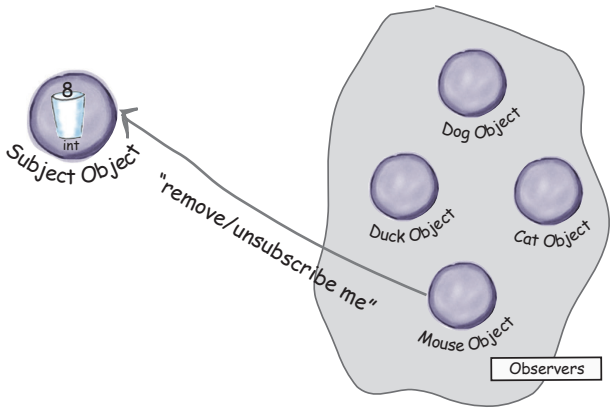
The Subject gets a new data value!

Now Duck and all the rest of the observers get a notification that the Subject has changed.



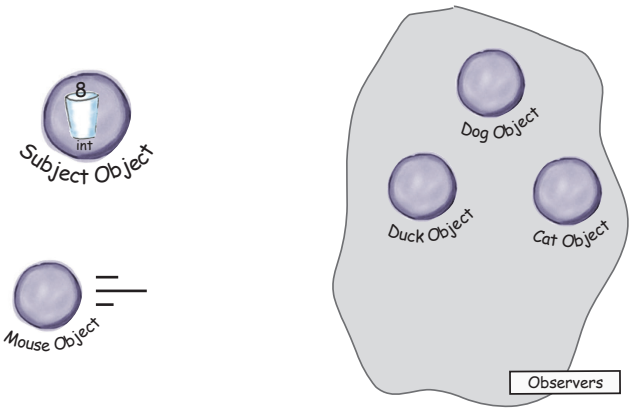
The Mouse object asks to be removed as an observer.

The Mouse object has been getting ints for ages and is tired of it, so he decides it's time to stop being an observer.



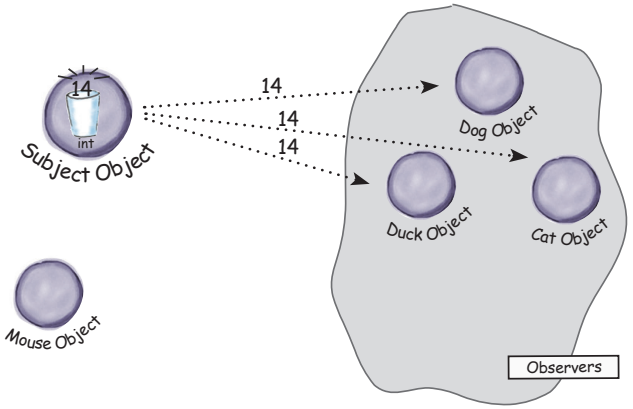
Mouse is outta here!

The Subject acknowledges the Mouse's request and removes him from the set of observers.



The Subject has another new int.

All the observers get another notification, except for the Mouse who is no longer included. Don't tell anyone, but the Mouse secretly misses those ints... maybe he'll ask to be an observer again some day.





Five-minute drama: a subject for observation

In today's skit, two enterprising software developers encounter a real live head hunter...



Software Developer #1



Headhunter/Subject



Software Developer #2



Subject

5 Meanwhile, for Lori and Jill life goes on; if a Java job comes along, they'll get notified. After all, they are observers.



6 Subject



7 Observer



Observer

Jill lands her own job!



8 Observer



9 Subject

Two weeks later...



Jill's loving life, and no longer an observer. She's also enjoying the nice fat signing bonus that she got because the company didn't have to pay a headhunter.

But what has become of our dear Lori? We hear she's beating the headhunter at his own game. She's not only still an observer, she's got her own call list now, and she is notifying her own observers. Lori's a subject and an observer all in one.



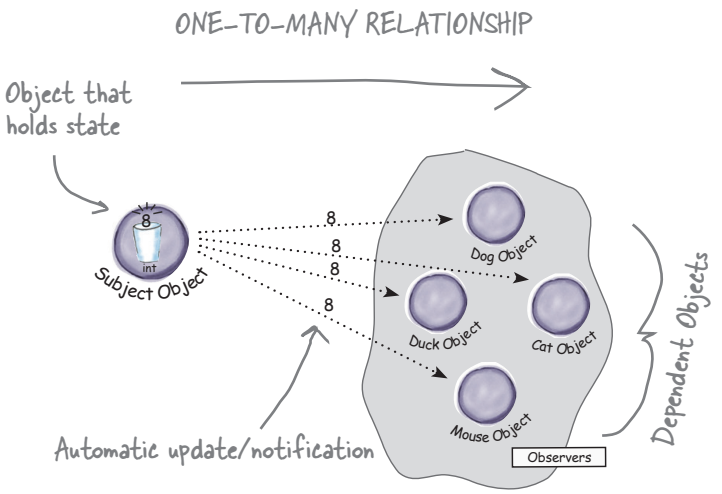
The Observer Pattern defined

A newspaper subscription, with its publisher and subscribers, is a good way to visualize the pattern.

In the real world, however, you'll typically see the Observer Pattern defined like this:

> **The Observer Pattern** defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

Let's relate this definition to how we've been thinking about the pattern:



The subject and observers define the one-to-many relationship. We have *one subject*, who notifies *many observers* when something in the subject changes. The observers *are dependent* on the subject—when the subject's state changes, the observers are notified.

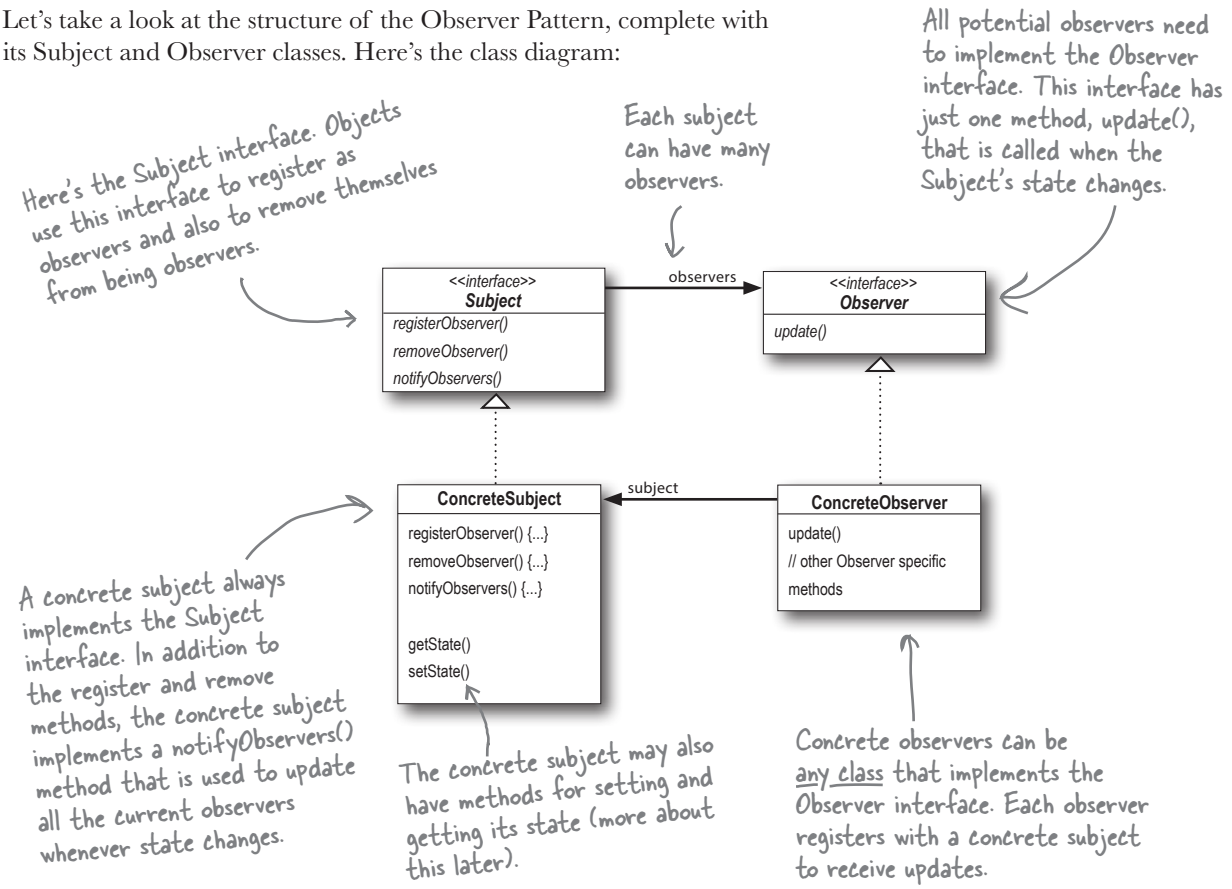
As you'll discover, there are a few different ways to implement the Observer Pattern, but most revolve around a class design that includes Subject and Observer interfaces.

The Observer Pattern defines a one-to-many relationship between a set of objects.

When the state of one object changes, all of its dependents are notified.

The Observer Pattern: the Class Diagram

Let's take a look at the structure of the Observer Pattern, complete with its Subject and Observer classes. Here's the class diagram:



there are no Dumb Questions

Q: What does this have to do with one-to-many relationships?

A: With the Observer Pattern, the Subject is the object that contains the state and controls it. So, there is ONE subject with state. The observers, on the other hand, use the state, even if they don't own it. There are many observers, and they rely on the Subject to tell them when its state changes. So there is a relationship between the ONE Subject to the MANY Observers.

Q: How does dependence come into this?

A: Because the subject is the sole owner of that data, the observers are dependent on the subject to update them when the data changes. This leads to a cleaner OO design than allowing many objects to control the same data.

Q: I've also heard of a Publish-Subscribe Pattern. Is that just another name for the Observer Pattern?

A: No, although they are related. The Publish-Subscribe pattern is a more complex pattern that allows subscribers to express interest in different types of messages and further separates publishers from subscribers. It is often used in middleware systems.



Guru and Student...

Guru: *Have we talked about loose coupling?*

Student: *Guru, I do not recall such a discussion.*

Guru: *Is a tightly woven basket stiff or flexible?*

Student: *Stiff, Guru.*

Guru: *And do stiff or flexible baskets tear or break less easily?*

Student: *A flexible basket tends to break less easily.*

Guru: *And in our software, might our designs break less easily if our objects are less tightly bound together?*

Student: *Guru, I see the truth of it. But what does it mean for objects to be less tightly bound?*

Guru: *We like to call it, loosely coupled.*

Student: *Ah!*

Guru: *We say a object is tightly coupled to another object when it is* **too** *dependent on that object.*

Student: *So a loosely coupled object can't depend on another object?*

Guru: *Think of nature; all living things depend on each other. Likewise, all objects depend on other objects. But a loosely coupled object doesn't know or care too much about the details of another object.*

Student: *But Guru, that doesn't sound like a good quality. Surely not knowing is worse than knowing.*

Guru: *You are doing well in your studies, but you have much to learn. By not knowing too much about other objects, we can create designs that can handle change better. Designs that have more flexibility, like the less tightly woven basket.*

Student: *Of course, I am sure you are right. Could you give me an example?*

Guru: *That is enough for today.*

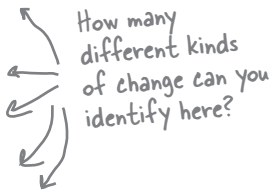
The Power of Loose Coupling

When two objects are *loosely coupled*, they can interact, but they typically have very little knowledge of each other. As we're going to see, loosely coupled designs often give us a lot of flexibility (more on that in a bit). And, as it turns out, the Observer Pattern is a great example of loose coupling. Let's walk through all the ways the pattern achieves loose coupling:

First, the only thing the subject knows about an observer is that it implements a certain interface (the Observer interface). It doesn't need to know the concrete class of the observer, what it does, or anything else about it.

We can add new observers at any time. Because the only thing the subject depends on is a list of objects that implement the Observer interface, we can add new observers whenever we want. In fact, we can replace any observer at runtime with another observer and the subject will keep purring along. Likewise, we can remove observers at any time.

We never need to modify the subject to add new types of observers. Let's say we have a new concrete class come along that needs to be an observer. We don't need to make any changes to the subject to accommodate the new class type; all we have to do is implement the Observer interface in the new class and register as an observer. The subject doesn't care; it will deliver notifications to any object that implements the Observer interface.



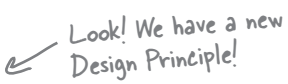
We can reuse subjects or observers independently of each other. If we have another use for a subject or an observer, we can easily reuse them because the two aren't tightly coupled.

Changes to either the subject or an observer will not affect the other. Because the two are loosely coupled, we are free to make changes to either, as long as the objects still meet their obligations to implement the Subject or Observer interfaces.



Design Principle

Strive for loosely coupled designs between objects that interact.



Loosely coupled designs allow us to build flexible OO systems that can handle change because they minimize the interdependency between objects.



Sharpen your pencil

Before moving on, try sketching out the classes you'll need to implement the Weather Station, including the WeatherData class and its display elements. Make sure your diagram shows how all the pieces fit together and also how another developer might implement her own display element.

If you need a little help, read the next page; your teammates are already talking about how to design the Weather Station.

Cubicle conversation

Back to the Weather Station project. Your teammates have already begun thinking through the problem...



Mary: Well, it helps to know we're using the Observer Pattern.

Sue: Right...but how do we apply it?

Mary: Hmm. Let's look at the definition again:

The Observer Pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Mary: That actually makes some sense when you think about it. Our WeatherData class is the "one," and our "many" is the various display elements that use the weather measurements.

Sue: That's right. The WeatherData class certainly has state...that's the temperature, humidity, and barometric pressure, and those definitely change.

Mary: Yup, and when those measurements change, we have to notify all the display elements so they can do whatever it is they are going to do with the measurements.

Sue: Cool, now I think I see how the Observer Pattern can be applied to our Weather Station problem.

Mary: There are still a few things to consider that I'm not sure I understand yet.

Sue: Like what?

Mary: For one thing, how do we get the weather measurements to the display elements?

Sue: Well, looking back at the picture of the Observer Pattern, if we make the WeatherData object the subject, and the display elements the observers, then the displays will register themselves with the WeatherData object in order to get the information they want, right?

Mary: Yes...and once the Weather Station knows about a display element, then it can just call a method to tell it about the measurements.

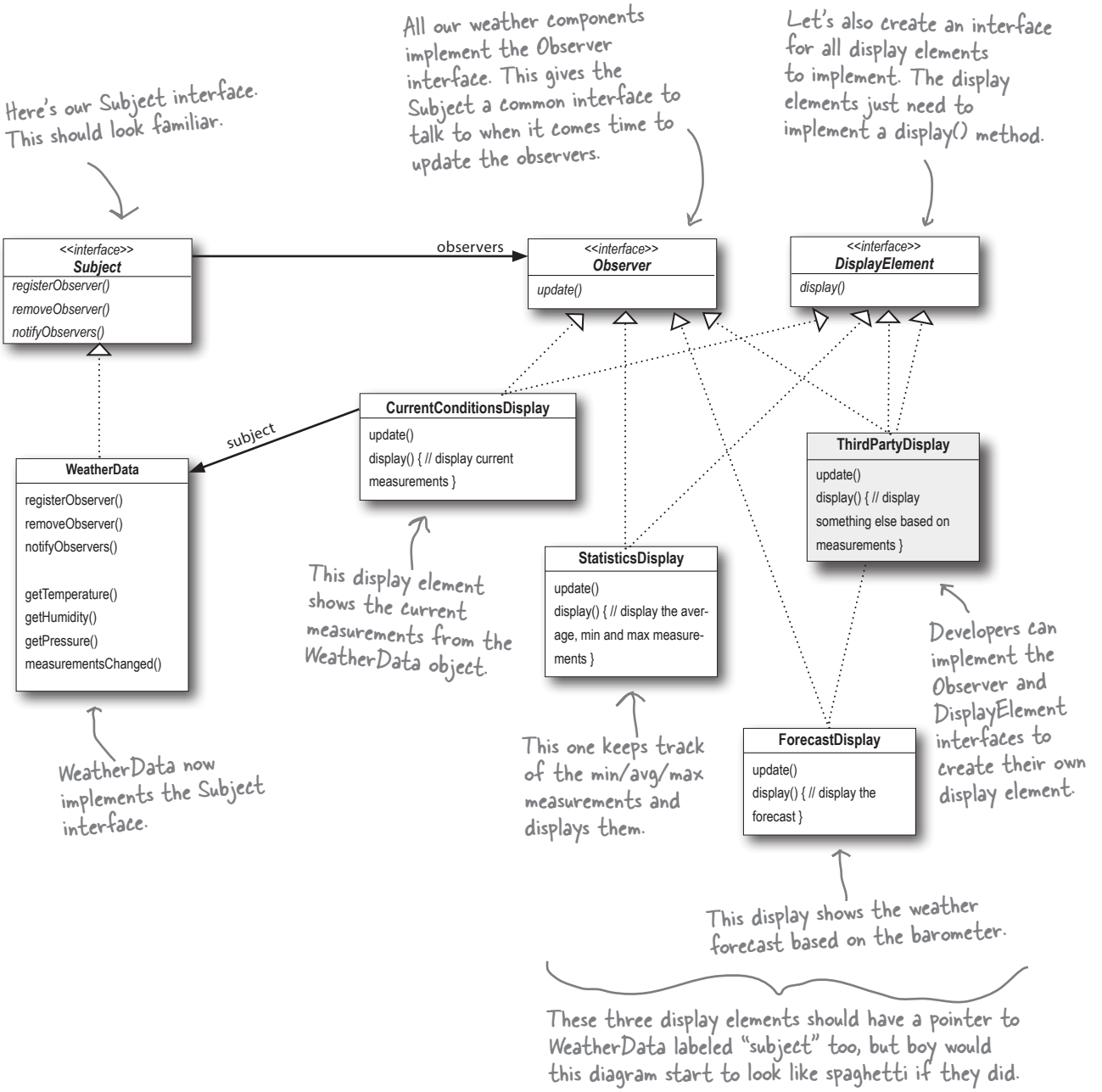
Sue: We gotta remember that every display element can be different...so I think that's where having a common interface comes in. Even though every component has a different type, they should all implement the same interface so that the WeatherData object will know how to send them the measurements.

Mary: I see what you mean. So every display will have, say, an update() method that WeatherData will call.

Sue: And update() is defined in a common interface that all the elements implement…

Designing the Weather Station

How does this diagram compare with yours?



Implementing the Weather Station

All right, we've had some great thinking from Mary and Sue (from a few pages back) and we've got a diagram that details the overall structure of our classes. So, let's get our implemention of the weather station underway. Let's start with the interfaces:

```
public interface Subject {
    public void registerObserver(Observer o);
    public void removeObserver(Observer o);
    public void notifyObservers();
}
```

Both of these methods take an Observer as an argument—that is, the Observer to be registered or removed.

This method is called to notify all observers when the Subject's state has changed.

```
public interface Observer {
    public void update(float temp, float humidity, float pressure);
}
```

These are the state values the Observers get from the Subject when a weather measurement changes.

The Observer interface is implemented by all observers, so they all have to implement the update() method. Here we're following Mary and Sue's lead and passing the measurements to the observers.

```
public interface DisplayElement {
    public void display();
}
```

The DisplayElement interface just includes one method, display(), that we will call when the display element needs to be displayed.



Mary and Sue thought that passing the measurements directly to the observers was the most straightforward method of updating state. Do you think this is wise? Hint: is this an area of the application that might change in the future? If it did change, would the change be well encapsulated, or would it require changes in many parts of the code?

Can you think of other ways to approach the problem of passing the updated state to the observers?

Don't worry; we'll come back to this design decision after we finish the initial implementation.

Implementing the Subject interface in WeatherData

Remember our first attempt at implementing the WeatherData class at the beginning of the chapter? You might want to refresh your memory. Now it's time to go back and do things with the Observer Pattern in mind:

REMEMBER: we don't provide import and package statements in the code listings. Get the complete source code from https://wickedlysmart.com/head-first-design-patterns

```
public class WeatherData implements Subject {
    private List<Observer> observers;
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherData() {
        observers = new ArrayList<Observer>();
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(temperature, humidity, pressure);
        }
    }

    public void measurementsChanged() {
        notifyObservers();
    }

    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }

    // other WeatherData methods here
}
```

WeatherData now implements the Subject interface.

We've added an ArrayList to hold the Observers, and we create it in the constructor.

When an observer registers, we just add it to the end of the list.

Likewise, when an observer wants to un-register, we just take it off the list.

Here's the fun part; this is where we tell all the observers about the state. Because they are all Observers, we know they all implement update(), so we know how to notify them.

Here we implement the Subject interface.

We notify the Observers when we get updated measurements from the Weather Station.

Okay, while we wanted to ship a nice little weather station with each book, the publisher wouldn't go for it. So, rather than reading actual weather data off a device, we're going to use this method to test our display elements. Or, for fun, you could write code to grab measurements off the web.

Now, let's build those display elements

Now that we've got our WeatherData class straightened out, it's time to build the display elements. Weather-O-Rama ordered three: the current conditions display, the statistics display, and the forecast display. Let's take a look at the current conditions display; once you have a good feel for this display element, check out the statistics and forecast displays in the code directory. You'll see they are very similar.

This display implements the Observer interface so it can get changes from the WeatherData object.

It also implements DisplayElement, because our API is going to require all display elements to implement this interface.

```
public class CurrentConditionsDisplay implements Observer, DisplayElement {
    private float temperature;
    private float humidity;
    private WeatherData weatherData;

    public CurrentConditionsDisplay(WeatherData weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    public void update(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }

    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and " + humidity + "% humidity");
    }
}
```

The constructor is passed the weatherData object (the Subject) and we use it to register the display as an observer.

When update() is called, we save the temp and humidity and call display().

The display() method just prints out the most recent temp and humidity.

there are no Dumb Questions

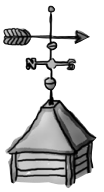
Q: Is update() the best place to call display()?

A: In this simple example it made sense to call display() when the values changed. However, you're right; there are much better ways to design the way the data gets displayed. We'll see this when we get to the Model-View-Controller pattern.

Q: Why did you store a reference to the WeatherData Subject? It doesn't look like you use it again after the constructor.

A: True, but in the future we may want to un-register ourselves as an observer and it would be handy to already have a reference to the subject.

Power up the Weather Station



1 First, let's create a test harness.

The Weather Station is ready to go. All we need is some code to glue everything together. We'll be adding some more displays and generalizing things in a bit. For now, here's our first attempt:

```
public class WeatherStation {

    public static void main(String[] args) {
        WeatherData weatherData = new WeatherData();

        CurrentConditionsDisplay currentDisplay =
            new CurrentConditionsDisplay(weatherData);
        StatisticsDisplay statisticsDisplay = new StatisticsDisplay(weatherData);
        ForecastDisplay forecastDisplay = new ForecastDisplay(weatherData);

        weatherData.setMeasurements(80, 65, 30.4f);
        weatherData.setMeasurements(82, 70, 29.2f);
        weatherData.setMeasurements(78, 90, 29.2f);
    }
}
```

First, create the WeatherData object.

If you don't want to download the code, you can comment out these two lines and run it.

Create the three displays and pass them the WeatherData object.

Simulate new weather measurements.

2 Run the code and let the Observer Pattern do its magic.

```
File Edit Window Help StormyWeather
%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
%
```

Sharpen your pencil

Johnny Hurricane, Weather-O-Rama's CEO, just called and they can't possibly ship without a Heat Index display element. Here are the details.

The heat index is an index that combines temperature and humidity to determine the apparent temperature (how hot it actually feels). To compute the heat index, you take the temperature, T, and the relative humidity, RH, and use this formula:

$$
\begin{aligned}
heatindex = {} & 16.923 + 1.85212 * 10^{-1} * T + 5.37941 * RH - 1.00254 * 10^{-1} * \\
& T * RH + 9.41695 * 10^{-3} * T^2 + 7.28898 * 10^{-3} * RH^2 + 3.45372 * \\
& 10^{-4} * T^2 * RH - 8.14971 * 10^{-4} * T * RH^2 + 1.02102 * 10^{-5} * T^2 * \\
& RH^2 - 3.8646 * 10^{-5} * T^3 + 2.91583 * 10^{-5} * RH^3 + 1.42721 * 10^{-6} \\
& * T^3 * RH + 1.97483 * 10^{-7} * T * RH^3 - 2.18429 * 10^{-8} * T^3 * RH^2 \\
& + 8.43296 * 10^{-10} * T^2 * RH^3 - 4.81975 * 10^{-11} * T^3 * RH^3
\end{aligned}
$$

So get typing!

Just kidding. Don't worry, you won't have to type that formula in; just create your own *HeatIndexDisplay.java* file and copy the formula from *heatindex.txt* into it.

You can get heatindex.txt from wickedlysmart.com.

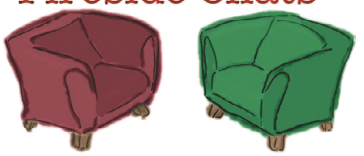
How does it work? You'd have to refer to *Head First Meteorology*, or try asking someone at the National Weather Service (or try a web search).

When you finish, your output should look like this:

Here's what changed in this output.

```
File  Edit  Window  Help  OverDaRainbow
%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Heat index is 82.95535
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Heat index is 86.90124
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
Heat index is 83.64967
%
```

Fireside Chats



Tonight's talk: **Subject and Observer spar over the right way to get state information to the Observer.**

Subject:

I'm glad we're finally getting a chance to chat in person.

Well, I do my job, don't I? I always tell you what's going on... Just because I don't really know who you are doesn't mean I don't care. And besides, I do know the most important thing about you—you implement the Observer interface.

Oh yeah, like what?

Well, excuuuse me. I have to send my state with my notifications so all you lazy Observers will know what happened!

Well...I guess that might work. I'd have to open myself up even more, though, to let all you Observers come in and get the state that you need. That might be kind of dangerous. I can't let you come in and just snoop around looking at everything I've got.

Observer:

Really? I thought you didn't care much about us Observers.

Yeah, but that's just a small part of who I am. Anyway, I know a lot more about you...

Well, you're always passing your state around to us Observers so we can see what's going on inside you. Which gets a little annoying at times...

Okay, wait just a minute here; first, we're not lazy, we just have other stuff to do in between your oh-so-important notifications, Mr. Subject, and second, why don't you let us come to you for the state we want rather than pushing it out to just everyone?

Subject:

Yes, I could let you **pull** my state. But won't that be less convenient for you? If you have to come to me every time you want something, you might have to make multiple method calls to get all the state you want. That's why I like **push** better...then you have everything you need in one notification.

Well, as I like to say, don't call us, we'll call you! But I'll give it some thought.

You never know, hell *could* freeze over.

Indeed.

Observer:

Why don't you just write some public getter methods that will let us pull out the state we need?

Don't be so pushy! There are so many different kinds of us Observers, there's no way you can anticipate everything we need. Just let us come to you to get the state we need. That way, if some of us only need a little bit of state, we aren't forced to get it all. It also makes things easier to modify later. Say, for example, you expand yourself and add some more state. If you use pull, you don't have to go around and change the update calls on every observer; you just need to change yourself to allow more getter methods to access our additional state.

I won't hold my breath.

I see, always the wise guy...

Looking for the Observer Pattern in the Wild



If you're curious about the Observer Pattern in JavaBeans, check out the PropertyChangeListener interface.

The Observer Pattern is one of the most common patterns in use, and you'll find plenty of examples of the pattern being used in many libraries and frameworks. If we look at the Java Development Kit (JDK), for instance, both the JavaBeans and Swing libraries make use of the Observer Pattern. The pattern's not limited to Java either; it's used in JavaScript's events and in Cocoa and Swift's Key-Value Observing protocol, to name a couple of other examples. One of the advantages of knowing design patterns is recognizing and quickly understanding the design motivation in your favorite libraries. Let's take a quick diversion into the Swing library to see how Observer is used.

The Swing library

You probably already know that Swing is Java's GUI toolkit for user interfaces. One of the most basic components of that toolkit is the JButton class. If you look up JButton's superclass, AbstractButton, you'll find that it has a lot of add/remove listener methods. These methods allow you to add and remove observers—or, as they are called in Swing, listeners—to listen for various types of events that occur on the Swing component. For instance, an ActionListener lets you "listen in" on any types of actions that might occur on a button, like a button press. You'll find various types of listeners all over the Swing API.

A little life-changing application

Okay, our application is pretty simple. You've got a button that says, "Should I do it?" and when you click on that button the listeners (observers) get to answer the question in any way they want. We're implementing two such listeners, called the AngelListener and the DevilListener. Here's how the application behaves:



Here's our fancy interface.



And here's the output when we click on the button.

```
File  Edit  Window  Help  HeMadeMeDoIt
%java SwingObserverExample
Come on, do it!
Don't do it, you might regret it!
%
```

Devil answer

Angel answer

Coding the life-changing application

This life-changing application requires very little code. All we need to do is create a JButton object, add it to a JFrame, and set up our listeners. We're going to use inner classes for the listeners, which is a common technique in Swing programming. If you aren't up on inner classes or Swing, you might want to review the Swing chapter in your favorite Java reference guide.

```
public class SwingObserverExample {
    JFrame frame;
    public static void main(String[] args) {
        SwingObserverExample example = new SwingObserverExample();
        example.go();
    }
    public void go() {
        frame = new JFrame();

        JButton button = new JButton("Should I do it?");
        button.addActionListener(new AngelListener());
        button.addActionListener(new DevilListener());

        // Set frame properties here
    }

    class AngelListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Don't do it, you might regret it!");
        }
    }

    class DevilListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Come on, do it!");
        }
    }
}
```

Simple Swing application that just creates a frame and throws a button in it.

Makes the devil and angel objects listeners (observers) of the button.

Code to set up the frame goes here.

Here are the class definitions for the observers, defined as inner classes (but they don't have to be).

Rather than update(), the actionPerformed() method gets called when the state in the subject (in this case the button) changes.



Serious Coding

Lambda expressions were added in Java 8. If you aren't familiar with them, don't worry about it; you can continue using inner classes for your Swing observers.

How about taking your use of the Observer Pattern even further? By using a lambda expression rather than an inner class, you can skip the step of creating an ActionListener object. With a lambda expression, we create a function object instead, and *the function object is the observer.* And, when you pass that function object to addActionListener(), Java ensures its signature matches actionPerformed(), the one method in the ActionListener interface.

Later, when the button is clicked, the button object notifies its observers—including the function objects created by the lambda expressions—that it's been clicked, and calls each listener's actionPerformed() method.

Let's take a look at how you'd use lambda expressions as observers to simplify our previous code:

The updated code, using lambda expressions:

```
public class SwingObserverExample {
    JFrame frame;
    public static void main(String[] args) {
        SwingObserverExample example = new SwingObserverExample();
        example.go();
    }
    public void go() {
        frame = new JFrame();

        JButton button = new JButton("Should I do it?");
        button.addActionListener(event ->
             System.out.println("Don't do it, you might regret it!"));
        button.addActionListener(event ->
             System.out.println("Come on, do it!"));

        // Set frame properties here
    }
}
```

We've replaced the AngelListener and DevilListener objects with lambda expressions that implement the same functionality that we had before.

When you click the button, the function objects created by the lambda expressions are notified and the method they implement is run.

Using lambda expressions makes this code a lot more concise.

We've removed the two ActionListener classes (DevilListener and AngelListener) completely.

For more on lambda expressions, check out the Java docs.

there are no Dumb Questions

Q: I thought Java had Observer and Observable classes?

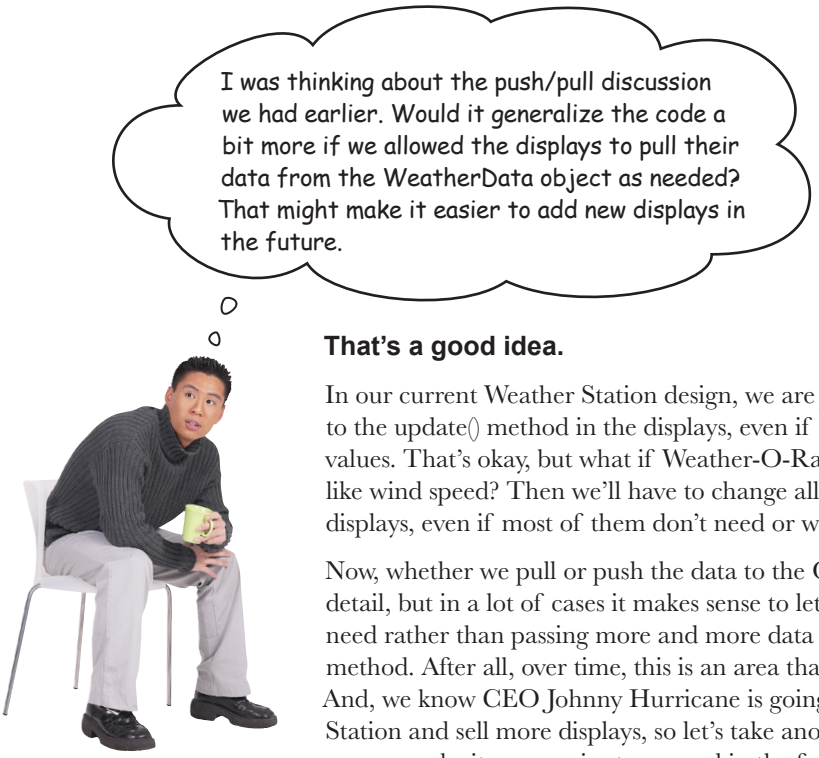
A: Good catch. Java used to provide an Observable class (the Subject) and an Observer interface, which you could use to help integrate the Observer Pattern in your code. The Observable class provided methods to add, delete, and notify observers, so that you didn't have to write that code. And the Observer interface provided an interface just like ours, with one update() method. These classes were deprecated in Java 9. Folks find it easier to support the basic Observer Pattern in their own code, or want something more robust, so the Observer/Observable classes are being phased out.

Q: Does Java offer other built-in support for Observer to replace those classes?

A: JavaBeans offers built-in support through PropertyChangeEvents that are generated when a Bean changes a particular kind of property, and sends notifications to PropertyChangeListeners. There are also related publisher/subscriber components in the Flow API for handling asynchronous streams.

Q: Should I expect notifications from a Subject to its Observers to arrive in a specific order?

A: With Java's implementations of Observer, the JDK developers specifically advise you to **not** depend on any specific notification order.



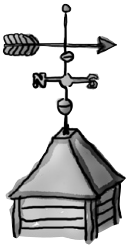
That's a good idea.

In our current Weather Station design, we are *pushing* all three pieces of data to the update() method in the displays, even if the displays don't need all these values. That's okay, but what if Weather-O-Rama adds another data value later, like wind speed? Then we'll have to change all the update() methods in all the displays, even if most of them don't need or want the wind speed data.

Now, whether we pull or push the data to the Observer is an implementation detail, but in a lot of cases it makes sense to let Observers retrieve the data they need rather than passing more and more data to them through the update() method. After all, over time, this is an area that may change and grow unwieldy. And, we know CEO Johnny Hurricane is going to want to expand the Weather Station and sell more displays, so let's take another pass at the design and see if we can make it even easier to expand in the future.

Updating the Weather Station code to allow Observers to *pull* the data they need is a pretty straightforward exercise. All we need to do is make sure the Subject has getter methods for its data, and then change our Observers to use them to pull the data that's appropriate for their needs. Let's do that.

Meanwhile, back at Weather-O-Rama



There's another way of handling the data in the Subject: we can rely on the Observers to pull it from the Subject as needed. Right now, when the Subject's data changes, we *push* the new values for temperature, humidity, and pressure to the Observers, by passing that data in the call to update().

Let's set things up so that when an Observer is notified of a change, it calls getter methods on the Subject to *pull* the values it needs.

To switch to using pull, we need to make a few small changes to our existing code.

For the Subject to send notifications...

1. We'll modify the notifyObservers() method in WeatherData to call the method update() in the Observers with no arguments:

```
public void notifyObservers() {
        for (Observer observer : observers) {
                observer.update();
        }
}
```

For an Observer to receive notifications...

1. Then we'll modify the Observer interface, changing the signature of the update() method so that it has no parameters:

```
public interface Observer {
        public void update();
}
```

2. And finally, we modify each concrete Observer to change the signature of its respective update() methds and get the weather data from the Subject using the WeatherData's getter methods. Here's the new code for the CurrentConditionsDisplay class:

```
public void update() {
        this.temperature = weatherData.getTemperature();
        this.humidity = weatherData.getHumidity();
        display();
}
```

Here we're using the Subject's getter methods that were supplied with the code in WeatherData from Weather-O-Rama.